# 1 Programmer's Model: Registers and ISA

The programmer's model is the set of registers visible to software and to the debugger. At this level there are **no variables**, only registers and memory: a C statement such as `x = x + 1` becomes "load `x` into a register, add 1, store it back".

The registers of the Cortex-M3 fall into two groups:

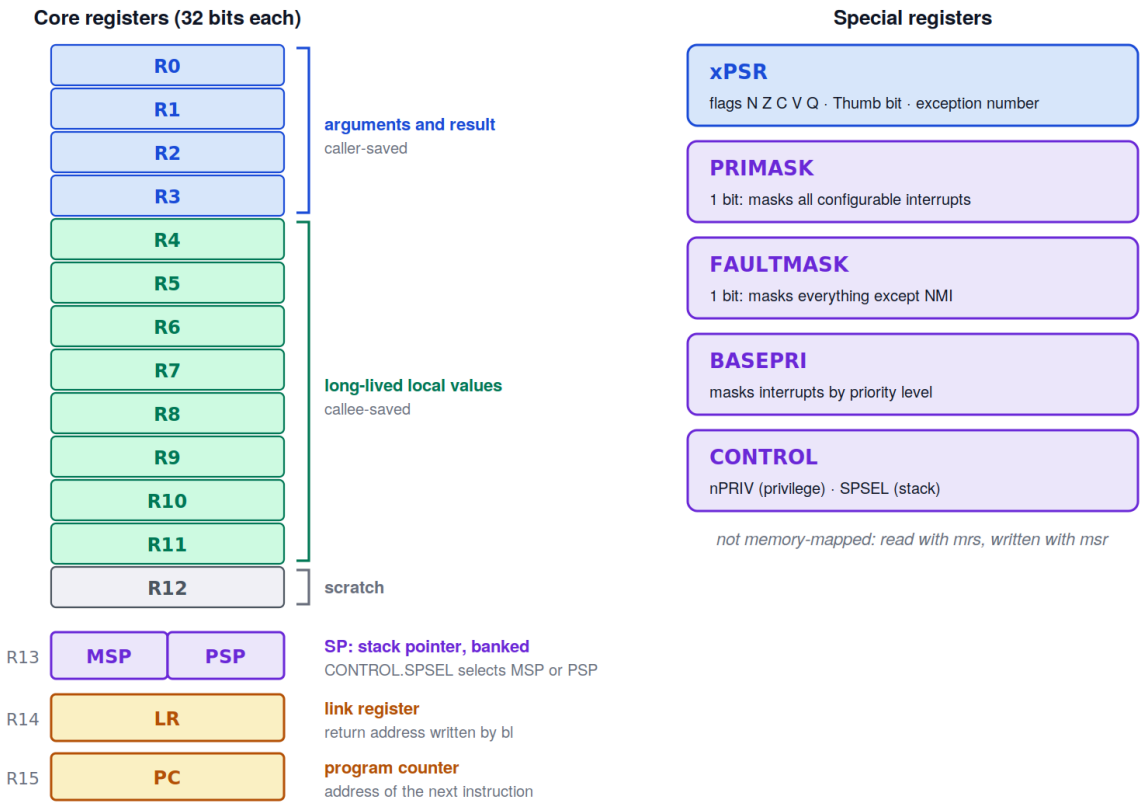
- **Core registers** ( `R0` – `R15` ): they are the operands of ordinary instructions, such as `add r0, r1, r2` or `ldr r3, [r0]` .
- **Special registers** ( `xPSR` , `PRIMASK` , `FAULTMASK` , `BASEPRI` , `CONTROL` ): they describe and control the state of the processor, and cannot be used as operands; they are read and written only with the dedicated instructions `mrs` and `msr` .

None of them has a memory address: they live inside the CPU. All of them are **32 bits** wide.

## 1.1 Core Registers

| Register | Name | Role |
|---|---|---|
| `R0` – `R12` | General purpose | Operands and addresses; `R0` – `R3` carry function arguments |
| `R13` | `SP` | Stack pointer: address of the top of the stack |
| `R14` | `LR` | Link register: return address after a function call |
| `R15` | `PC` | Program counter: address of the next instruction |
| — | `xPSR` | Program status: flags, Thumb bit, current exception number |

The stack is **full descending**: it grows towards lower addresses, and `SP` points to the last occupied word. A push first decrements `SP` by 4 and then writes at the new `SP` . `R13` is a banked name: two physical stack pointers exist ( `MSP` and `PSP` ), and `SP` always refers to the one currently selected (see the chapter on execution modes and exceptions).




**Cortex-M3 registers:** *thirteen general-purpose registers with their AAPCS roles, the banked stack pointer, the link register, the program counter, and the special registers accessed with mrs and msr.*

## 1.2 Special Registers

Special registers hold the **state of the processor** rather than data. They cannot be used as operands of ordinary instructions such as `add` or `ldr`, and they have **no memory address**. They are moved to and from a core register with two dedicated instructions:

```
mrs r0, primask     @ move register from special: r0 = PRIMASK
msr control, r0     @ move special from register: CONTROL = r0
```

The names follow the direction of the copy: `mrs` copies a **special** register into a core **register**, `msr` copies a core **register** into a **special** register. To change a single bit, the usual read-modify-write sequence is needed: `mrs` to read, `orr` or `bic` to modify, `msr` to write back.

| Special register | Content | Described in |
|---|---|---|
| `xPSR` | Program status: flags, Thumb bit, current exception number | The Program Status Register xPSR |
| `CONTROL` | Thread-mode privilege and stack selection | The CONTROL Register |
| `PRIMASK` | 1 bit: masks every exception with configurable priority | The PRIMASK Register |
| `FAULTMASK` | 1 bit: also masks HardFault; only NMI is taken | — |

| Special register | Content | Described in |
|---|---|---|
| `BASEPRI` | Masks exceptions whose priority value is greater than or equal to it | — |

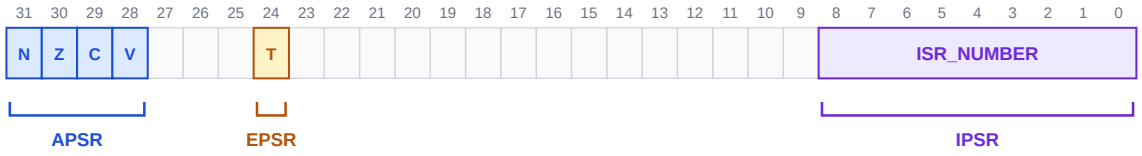
`FAULTMASK` and `BASEPRI` are listed for completeness: the firmware described in these notes does not use them.

> **WRITING `CONTROL`, `PRIMASK`, `FAULTMASK`, AND `BASEPRI` IS RESERVED TO PRIVILEGED CODE. IN UNPRIVILEGED CODE THE WRITE IS SILENTLY IGNORED: NO FAULT IS RAISED, THE REGISTER SIMPLY KEEPS ITS VALUE.**

### 1.2.1 The Program Status Register xPSR

`xPSR` holds neither data nor addresses: it **describes** the current **state of the processor**. The name reflects that it is the union of three sub-registers occupying disjoint bits of the same 32-bit word.

| Sub-register | Bits | Content |
|---|---|---|
| `APSR` (Application) | 31–28 | Flags `N` (negative), `Z` (zero), `C` (carry), `V` (overflow) |
| `EPSR` (Execution) | 24 | `T`: Thumb state, always 1 on Cortex-M |
| `IPSR` (Interrupt) | 8–0 | Number of the exception being handled, 0 if none |




APSR: condition flags · EPSR: Thumb state (always 1) · IPSR: running exception (0 = Thread mode)

**Figure 1.2:** *Bit layout of xPSR: N, Z, C, V flags in APSR, Thumb bit in EPSR, exception number in IPSR.*

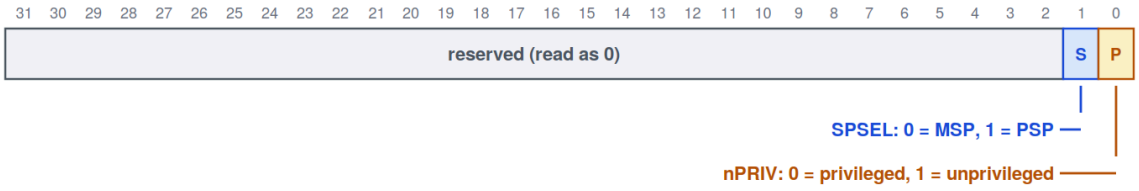
The flags are written by comparison and arithmetic instructions and read by conditional branches; this is how every `if` and loop condition is implemented at machine level.

> **FLAGS IN ACTION `CMP R1, R2` COMPUTES `R1 - R2`, DISCARDS THE RESULT, AND UPDATES ONLY THE FLAGS: `Z = 1` IF THE OPERANDS ARE EQUAL, `C = 1` IF `R1 >= R2` AS UNSIGNED VALUES. THE FOLLOWING `BHS DONE` READS `C` AND BRANCHES IF IT IS 1.**

`IPSR` identifies the execution mode: 0 means Thread mode (ordinary code), any other value is the number of the exception whose handler is running.

### 1.2.2 The CONTROL Register

`CONTROL` decides **how ordinary code runs**: with or without privileges, and on which of the two stacks. Only its **two lowest bits are used** on the Cortex-M3; the other bits are reserved and read as 0.




both bits act only in Thread mode; Handler mode is always privileged and always uses MSP

**CONTROL register:** *bit 1 (SPSEL) selects the Thread-mode stack, bit 0 (nPRIV) the Thread-mode privilege; bits 31–2 are reserved.*

| Bit | Name | Meaning |
|---|---|---|
| 0 | `nPRIV` | 0 = privileged, 1 = unprivileged (Thread mode only) |
| 1 | `SPSEL` | Thread-mode stack pointer: 0 = `MSP`, 1 = `PSP` |

The two bits give four possible values:

| `CONTROL` | Thread privilege | Thread stack | Typical use |
|---|---|---|---|
| `0` | Privileged | `MSP` | State after reset: boot code and kernel |
| `1` | Unprivileged | `MSP` | Rarely used |
| `2` | Privileged | `PSP` | Rarely used |
| `3` | Unprivileged | `PSP` | Application task with its own stack |

Both bits act only in **Thread mode**, the mode of ordinary code. Exception handlers run in Handler mode, which is always privileged and always uses `MSP`, whatever `CONTROL` contains. The meaning of the two modes, of privilege, and of the two stacks is described in the chapter on execution modes and exceptions.

`CONTROL` is changed with a read-modify-write on a special register, followed by an `isb` instruction:

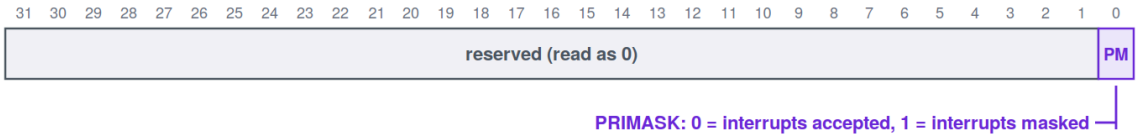
```
mrs r0, control     @ r0 = CONTROL
orr r0, r0, #1      @ set nPRIV
msr control, r0     @ CONTROL = r0: Thread mode is now unprivileged
isb                 @ apply the new setting to the following instructions
```

The `isb` (instruction synchronization barrier) is needed because the CPU has already fetched the next instructions under the old setting; `isb` discards them, so that every following instruction runs with the new value.

> **A ONE-WAY CHANGE ONLY PRIVILEGED CODE CAN WRITE `CONTROL`. ONCE THREAD MODE HAS BECOME UNPRIVILEGED, A `MSR CONTROL` THAT TRIES TO CLEAR `NPRIV` IS SILENTLY IGNORED: THE ONLY WAYS BACK TO PRIVILEGED EXECUTION ARE A RESET OR AN EXCEPTION, WHOSE HANDLER RUNS PRIVILEGED IN HANDLER MODE.**

### 1.2.3 The PRIMASK Register

`PRIMASK` is a one-bit register that **switches interrupts off and on**. Only bit 0 is used; the other bits are reserved and read as 0.




when set, only NMI and HardFault are taken; masked interrupts stay pending

**PRIMASK register:** *bit 0 masks every exception with configurable priority; bits 31–1 are reserved.*

| `PRIMASK` | Effect |
|---|---|
| `0` | Interrupts are accepted normally (value after reset) |
| `1` | Every exception with configurable priority is held back: device interrupts, SysTick, PendSV, SVCall. Only NMI and HardFault can still be taken |

Masked interrupts are **not lost**: they remain *pending*, and the CPU takes them as soon as `PRIMASK` returns to 0. Exceptions, priorities, and pending state are described in the chapter on execution modes and exceptions.

> ⚠ `SVC` WITH INTERRUPTS MASKED SVCALL IS THE EXCEPTION THAT CANNOT WAIT: `SVC` IS A SYNCHRONOUS REQUEST AND CANNOT STAY PENDING. EXECUTED WHILE `PRIMASK` IS 1, IT ESCALATES TO HARDFAULT.

`PRIMASK` is usually changed with two dedicated instructions, which do not need a core register:

| Assembly | C (CMSIS) | Effect |
|---|---|---|
| `cpsid i` | `__disable_irq();` | `PRIMASK = 1` : interrupts masked |
| `cpsie i` | `__enable_irq();` | `PRIMASK = 0` : interrupts accepted |

The main use of `PRIMASK` is a *critical section*: a short piece of code that must run without being interrupted, for example while updating a variable that an interrupt handler also uses. Boot code also masks interrupts at the start and unmasks them only once the vector table, handlers, and priorities are configured.

A critical section must restore the **previous** value of `PRIMASK`, not simply unmask interrupts. If a critical section runs inside another one, an unconditional `cpsie i` at its end would re-enable interrupts while the outer section is still running. The safe pattern saves `PRIMASK` with `mrs` and restores it with `msr`:

```
mrs   r0, primask   @ save the previous value (0 or 1)
cpsid i             @ mask interrupts
...                 @ critical section
msr   primask, r0   @ restore the previous value
```

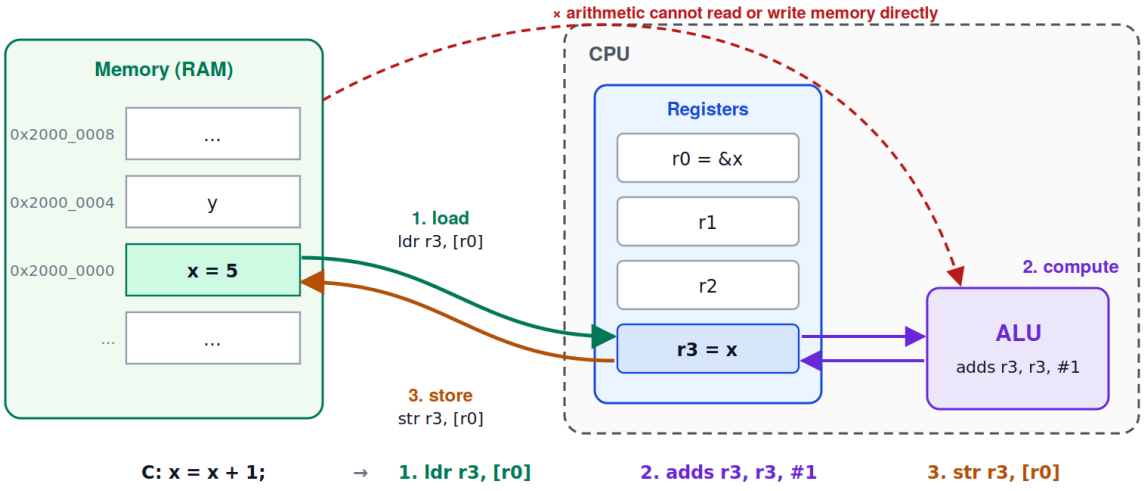
> 🔥 ONLY PRIVILEGED CODE CAN CHANGE `PRIMASK`. IN UNPRIVILEGED CODE `CPSID I`, `CPSIE I`, AND `MSR PRIMASK` ARE SILENTLY IGNORED, SO AN APPLICATION TASK CANNOT SWITCH INTERRUPTS OFF.

## 1.3 Load/Store Architecture

The Cortex-M3 has a **load/store instruction set**: arithmetic and logic operate **only on registers**, and memory is accessed exclusively through load ( `ldr` , memory to register) and store ( `str` , register to memory) instructions.

**Every operation** on a variable therefore **passes through a register**.

Two rules help to read every instruction below. The first operand is usually the destination ( `ldr r3, …` writes `r3` ), except in stores, where the first register is the value to save. Square brackets `[ ]` always mean "**the memory at this address**".




**Load/store architecture:** *memory is reached only through loads and stores; arithmetic works on registers. The C statement x = x + 1 needs three instructions.*

## 1.3.1 Load/Store Instructions

Load and store instructions are the only instructions that **access memory**. A *load* copies a value from memory into a register, a *store* copies a register into memory. The address is always taken from a register, written in square brackets, and can be adjusted by an offset ( `[r0, #7]` ) or updated after the access ( `[r0], #4` ). Suffixes select the size of the access: no suffix for a 32-bit word, `b` for a byte, `h` for a halfword (16 bits).

| Assembly | C equivalent | Explanation |
|---|---|---|
| `ldr r3, [r0]` | `r3 = *r0;` | *Load register*: read the 32-bit word at address `r0` into `r3` |
| `str r3, [r1]` | `*r1 = r3;` | *Store register*: write `r3` to memory at address `r1` |
| `ldr r3, [r0], #4` | `r3 = *r0; r0 += 4;` | Load, then add 4 to `r0` so it points to the next word: used to walk arrays |
| `strb r1, [r0, #7]` | `((uint8_t *)r0)[7] = r1;` | *Store byte*: write only the low byte of `r1` at `r0 + 7`; `r0` does not change |
| `ldr r0, =0x20400000` | `r0 = 0x20400000;` | Pseudo-instruction: load a 32-bit constant written as a number, through the literal pool |
| `ldr r0, =_estack` | `r0 = (uint32_t)&_estack;` | Same as above, with the value given by a symbol: the linker replaces `_estack` with `0x20400000` |

### 1.3.1.1 LOADING A 32-BIT CONSTANT: THE LITERAL POOL

A Thumb instruction is 16 or 32 bits long. These bits must hold the operation and the registers, so there is no space left for a full 32-bit value such as the address `0x20400000`. The constant must therefore be stored in

memory and loaded from there.

The pseudo-instruction `ldr r0, =0x20400000` does this automatically. The assembler:

1. writes the value `0x20400000` in memory, as data, just after the code; this small area of constants is called the *literal pool*;
2. replaces the pseudo-instruction with a normal load that reads that word. The load gives the position of the word as a distance from the current instruction, so a disassembler shows it as `ldr r0, [pc, #4]`, which means "read the word 4 bytes ahead of here".

```
0x48:  ldr r0, [pc, #4]     reads the word at 0x50 into r0
0x4A:  ...                  rest of the code
0x4E:  b   elsewhere        the CPU jumps away before reaching 0x50
0x50:  0x20400000           literal pool: a constant, not an instruction
```

The CPU must never execute the literal pool: if it did, it would read the bytes of `0x20400000` as an instruction. The directive `.ltorg` tells the assembler where to put the pool, and it is placed where execution never arrives, for example right after an unconditional branch.

### 1.3.2 Data-Processing Instructions

Data-processing instructions compute on registers: arithmetic (`add`, `sub`), logic (`and`, `orr`), bit shifts, copies between registers, and comparisons. They **never touch memory**. The destination is the first operand; the last operand can be a register or a small constant written with `#`. A trailing `s` (as in `movs`) also updates the flags of `xPSR`, and comparison instructions do nothing else.

| Assembly | C equivalent | Explanation |
|---|---|---|
| `movs r3, #0` | `r3 = 0;` | *Move*: put a small constant in `r3` and update the flags |
| `mov sp, r0` | `sp = r0;` | *Move*: copy one register into another, here into the stack pointer |
| `orr r1, r1, #0x10` | `r1 \|= 0x10;` | Bitwise *OR*: set bit 4 of `r1`, leave the other bits unchanged |
| `cmp r1, r2` | (compute `r1 - r2`) | *Compare*: subtract, update the flags with the result, then discard the result |

### 1.3.3 Branch Instructions

Branch instructions **change the flow of execution** by **loading a new address into the program counter**. An unconditional branch always jumps; a conditional branch jumps only if the flags satisfy its condition, which is written as a suffix (`eq`, `ne`, `hs`, `lt`…) and usually tests the result of a previous `cmp`. A *branch with link* also saves the return address in `lr`, which is how function calls work.

| Assembly | C equivalent | Explanation |
|---|---|---|
| `b label` | `goto label;` | *Branch*: always jump to `label` |
| `bhs done` | `if (r1 >= r2) goto done;` | *Branch if higher or same*: jump only if the last `cmp` found `r1 >= r2` (unsigned) |

| Assembly | C equivalent | Explanation |
|---|---|---|
| `bl main` | `main();` | *Branch with link*: save the return address in `lr`, then jump; return is `bx lr` |

#### 1.3.3.1 WRITING A LOOP: COMPARE AND BRANCH

Assembly has no `while` or `for`. A loop is built from three pieces: a compare, a conditional branch that leaves the loop, and an unconditional branch that goes back to the start.

The C loop:

```
while ( r1 < r2 ) {
    /* body */
}
```

becomes:

```
loop:   cmp   r1, r2        @ 1. compare r1 with r2
        bhs   done          @ 2. if r1 >= r2, jump to done (exit)
        ...                 @ 3. body of the loop
        b     loop          @ 4. jump back to step 1
done:                       @ code after the loop
```

`loop:` and `done:` are *labels*: names given to code addresses, used as targets of branches.

- **Step 1**: `cmp` subtracts `r2` from `r1`, throws the result away, and only updates the flags, which record how the comparison went.
- **Step 2**: `bhs` (*branch if higher or same*) looks at the flags and jumps to `done` if `r1 >= r2`. It tests the opposite of the C condition, because it decides when to *leave* the loop.
- **Step 3**: if the branch is not taken, the CPU simply continues with the next instructions, which are the body of the loop.
- **Step 4**: `b` always jumps back to `loop`, so the comparison is repeated before every new iteration.

### 1.3.4 Special Registers Access Instructions

Special registers cannot be used as operands of ordinary instructions, so a separate group of instructions moves them to and from core registers. `cpsid` and `cpsie` are shortcuts that change only `PRIMASK`, without going through a core register. Writing a special register is allowed only in privileged code; in unprivileged code the write is silently ignored.

| Assembly | C equivalent | Explanation |
|---|---|---|
| `mrs r0, control` | `r0 = CONTROL;` | *Move register from special*: copy a special register into a core register |
| `msr control, r0` | `CONTROL = r0;` | *Move special from register*: copy a core register into a special register |
| `mrs r0, ipsr` | `r0 = IPSR;` | Read one part of `xPSR`, here the number of the running exception |

| Assembly | C equivalent | Explanation |
|---|---|---|
| `cpsid i` | `__disable_irq();` | *Change processor state, interrupts disable*: set `PRIMASK` to 1, masking interrupts |
| `cpsie i` | `__enable_irq();` | *Change processor state, interrupts enable*: clear `PRIMASK` to 0 |

## 1.4 Thumb State and the Thumb Bit

The Cortex-M executes only the Thumb-2 instruction set, a mix of 16-bit and 32-bit instructions. This has a side effect on code addresses:

- **Bit 0 is free**: instructions always start at an even address, so bit 0 of a code address is always 0 and carries no information.
- **ARM reuses it as the Thumb bit**: every code pointer the CPU jumps to (a vector table entry, a function pointer, the target of `bx`) must have bit 0 set to 1.
- **The CPU clears it before jumping**: it copies the bit into `xPSR.T`, sets it to 0, and jumps to the even address.
- **Example**: a handler located at `0x40` is stored in the vector table as `0x41`.

> ⚠ **THUMB BIT EQUAL TO 0 BRANCHING TO AN ADDRESS WITH BIT 0 CLEAR REQUESTS ARM STATE, WHICH THE CORTEX-M DOES NOT IMPLEMENT. THE RESULT IS A USAGEFAULT (`INVSTATE`), NOT AN ORDINARY CRASH AT THE TARGET ADDRESS.**

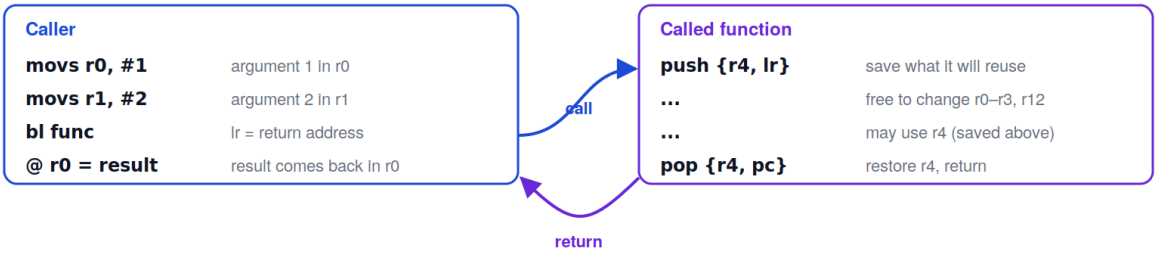
## 1.5 Calling Convention (AAPCS)

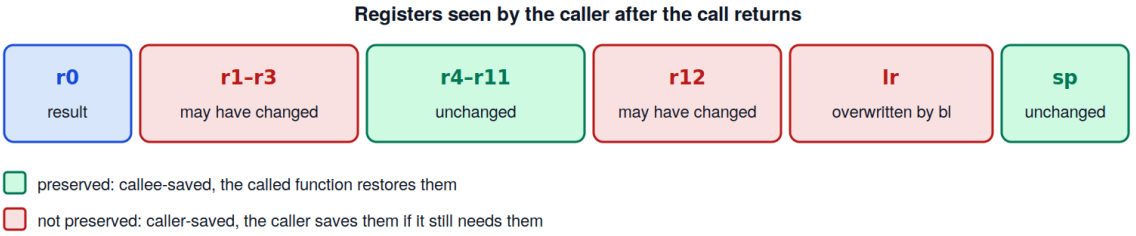
The **AAPCS (Procedure Call Standard for the Arm Architecture)** defines how functions **exchange arguments** and **which registers** they must preserve. Compiled C and hand-written assembly interoperate only if both follow it, and the exception model is designed around it.

| Register | Role in a call | Preserved across the call |
|---|---|---|
| `r0` – `r3` | First four arguments; `r0` also returns the result | No (caller-saved) |
| `r4` – `r11` | Long-lived local values | Yes (callee-saved) |
| `r12` | Scratch | No (caller-saved) |
| `lr` | Return address written by `bl` | No; a non-leaf function saves it |
| `sp` | Multiple of 8 at every public call | Yes |

Registers are shared: when a function calls another one, the called function may overwrite values the caller still needs. The AAPCS splits the responsibility for saving them:

- **`r0` – `r3` and `r12` are the caller's problem.** The called function may change them freely. If the caller still needs one of these values after the call, it must save it before calling.
- **`r4` – `r11` are the called function's problem.** The caller finds them unchanged after the call. A function that wants to use `r4` must save its old value at entry and restore it before returning, typically with `push {r4, lr}` at entry and `pop {r4, pc}` at exit; the same `pop` also returns, because it loads the saved `lr` into `pc`.
- **A leaf function may save nothing.** A *leaf function* calls no other function, so `lr` is never overwritten. If it also uses only `r0` – `r3` and `r12`, it has nothing to restore and does not touch the stack at all.




**AAPCS function call:** *arguments travel in r0–r3 and the result in r0; the called function restores r4–r11 and sp, while r0–r3, r12, and lr may be changed by the call.*

### A LEAF FUNCTION: `ADD3` CONSIDER A MINIMAL C FUNCTION THAT ADDS THREE NUMBERS:

```
int add3( int a, int b, int c )
{
    return a + b + c;
}
```

Following the AAPCS, the arguments `a`, `b`, `c` arrive in `r0`, `r1`, `r2`, and the result must be returned in `r0`. The compiler translates the function into three instructions:

```
add3:
    adds r0, r0, r1     @ r0 = a + b
    adds r0, r0, r2     @ r0 = r0 + c
    bx   lr             @ return; the result is in r0
```

A call such as `r = add3( 1, 2, 3 );` places the arguments in registers and jumps to the function:

```
    movs r0, #1         @ a = 1
    movs r1, #2         @ b = 2
    movs r2, #3         @ c = 3
    bl   add3           @ lr = return address, jump to add3
                        @ back here: r0 = 6
```

`add3` is a *leaf function*: it calls no other function, so `lr` is never overwritten, and it uses only `r0` – `r2`, which are caller-saved. It therefore saves nothing: no `push`, no `pop`, no use of the stack, and it returns with a simple `bx lr`.